WCAG 2.2 · W3C

Accesibilidad web: botones, etiquetas y teclado



De un «Pagar» que solo funciona con mouse a un checkout que cualquiera completa: con teclado, lector de pantalla o zoom al 200 %.

Chamba Lab · 2026



Accesibilidad: que todos puedan usar lo que construyes

Con mouse, teclado, lector o zoom.

- **WCAG**: las pautas del W3C para medirla, con criterios numerados
- 2.2 es Recomendación W3C desde octubre de 2023 (edición vigente: 12-12-2024)
- WCAG 3 sigue en borrador y no reemplaza a la 2.2
- El ejemplo de todo el manual: el checkout de la tiendita de Rakim

Para quién: cada persona llega con algo distinto



El búho: 72 años, navega con zoom al 200 %

Lector de pantalla	NVDA, JAWS, VoiceOver, TalkBack. **Necesita nombre y rol**: «Pagar, botón».
Solo teclado	Tab, Shift+Tab, Enter, Espacio. La oveja, con el brazo enyesado. **Necesita foco visible.**
Zoom y lupa	El búho, al 200 %. **Necesita que nada se corte** ni quede tapado.
Baja visión o daltonismo	**Necesita contraste** y no depender solo del color.

El 16 % de la población mundial vive con una discapacidad importante (OMS). En el Perú, unos 3,2 millones de personas (INEI, Censos 2017).

Los 4 principios: POUR, el esqueleto de WCAG



Se percibe

Por algún sentido: el logo de Yape tiene `alt`, el texto tiene contraste.

Se opera

Sin exigir lo que no puedes hacer: «Pagar» responde al teclado.

Se entiende

El campo dice qué pide y el error dice cómo arreglarlo.

Es robusto

HTML que cualquier navegador y lector interpreta igual.

Niveles: A, AA y AAA, de lo mínimo a lo ideal

A · el piso

Sin esto, alguien no puede usar la página: teclado, `alt`, etiquetas, nombre y rol.

`1.1.1` `2.1.1` `3.3.2` `4.1.2`

OBLIGATORIO

AA · la meta

Cumple todo A y AA: contraste 4,5:1, foco visible, zoom al 200 %, objetivos de 24 px.

`1.4.3` `1.4.4` `2.4.7` `2.5.8`

LO QUE PIDEN CONTRATOS Y LEYES

AAA · el extra

Contraste 7:1, objetivos de 44 px. El W3C no recomienda exigirlo a un sitio entero.

`1.4.6` `2.5.5` `2.4.13`

POR PANTALLA, CUANDO SE PUEDE

Cómo funciona: tu HTML se vuelve lo que el lector dice




```
<form action="/pagar"
      method="post">
  <!— correo, tarjeta… —>
  <button type="submit">
    Pagar S/ 89.90
  </button>
</form>
```

Con un solo elemento ya tienes rol, foco con Tab, Enter y Espacio, y el envío del formulario.

Regla 1: HTML semántico antes que ARIA

`<button>`	**Hace algo aquí:** pagar, abrir, enviar.
`<a href>`	**Te lleva** a otra página o sección.
`<label>`	**Nombra un campo** del formulario.

`<h1>…<h3>`	**Titula en orden:** el lector salta de título en título.
`<main>`	**Marca las zonas** (con `<nav>`, `<header>`) para saltar directo.
`alt`	**Describe la imagen** que informa; `alt=""` si solo decora.

«Sin ARIA es mejor que con ARIA mala» (APG). Un `role="button"` promete un botón, pero no le da foco ni teclado.

Regla 2: cada campo con su `<label>`

checkout.html

```
<label for="correo">Correo</label>
<input id="correo" type="email"
       autocomplete="email"
       placeholder="ana@correo.pe">

<label for="tarjeta">Número de tarjeta</label>
<input id="tarjeta" inputmode="numeric"
       autocomplete="cc-number"
       aria-describedby="ayuda">
<p id="ayuda">Sin espacios ni guiones</p>
```

`for` = `id`: el lector lee la etiqueta y un clic en ella enfoca el campo.

`placeholder` es solo un ejemplo: se borra al escribir y el lector no lo anuncia por defecto.

`autocomplete` deja que el navegador rellene; `aria-describedby` suma la ayuda.

Regla 3: el foco siempre se ve

- **2.4.7 (AA):** quien usa teclado ve dónde está
- `:focus-visible` marca el foco al tabular y no al hacer clic
- **2.4.11 (AA, nuevo):** la barra fija o el banner de cookies no pueden taparlo del todo
- El indicador también necesita contraste 3:1

El tucán puso `outline: none` «porque se veía feo».

```
/* nunca outline: none
   sin poner otro estilo */
:focus-visible {
  outline: 3px solid #1d4ed8;
  outline-offset: 2px;
}

/* barra fija de 80px abajo */
html {
  scroll-padding-bottom: 96px;
}
```

Técnica C43 del W3C: al tabular, el navegador deja 96 px libres y la barra no tapa el campo.

Regla 4: contraste que se lee al sol



El tucán: «el gris clarito es más elegante»

Correo	2,5:1 FALLA	Correo	4,8:1 PASA AA
Pagar	2,3:1 FALLA	Pagar	5,0:1 PASA AA

- **1.4.3 (AA):** 4,5:1 para texto normal; 3:1 si es grande (18 pt, o 14 pt en negrita)
- **1.4.11 (AA):** 3:1 para bordes de campos, iconos e indicador de foco
- El error no va solo en rojo: suma un texto que lo diga

Lo nuevo de 2.2: seis criterios A y AA en el checkout

2.4.11 · AA

Foco no tapado

El banner de cookies no esconde el botón que tiene el foco.

2.5.8 · AA

Tamaño del objetivo

Mínimo 24×24 px, o espacio alrededor: los «−» y «+» de cantidad.

2.5.7 · AA

Arrastrar

Lo que se arrastra también se hace con un clic: el deslizador de cuotas.

3.3.7 · A

Entrada redundante

No pidas dos veces lo mismo: «usar la dirección de envío».

3.3.8 · AA

Acceso sin acertijos

Deja pegar la contraseña y usar el gestor de contraseñas.

3.2.6 · A

Ayuda consistente

El enlace de ayuda, en el mismo lugar en cada paso.

También suma tres AAA (2.4.12, 2.4.13 y 3.3.9) y retira 4.1.1 Parsing.



¿Por qué el búho no puede pagar si el botón se ve perfecto?

Con mouse paga. Con Tab, el foco pasa de largo.

El botón pintado: un <div onclick> que dice «Pagar»

Un no recibe foco ni escucha Enter. El lector lo lee como texto suelto, sin «botón».

- Falla 2.1.1 Teclado y 4.1.2 Nombre, función, valor: los dos son nivel A
- El búho y la oveja se quedan sin pagar
- En la demo se ve idéntico: nadie lo nota

```
<form action="/pagar" method="post">
  <!— correo, tarjeta… —>
  <div class="btn-pagar" onclick="pagar()">
    Pagar S/ 89.90
  </div>
</form>

<script>
  function pagar() {
    document.querySelector('form').submit();
  }
</script>
```

Dos arreglos: imitar el botón o usar el de verdad

El parche: div + ARIA + JS

```
<div role="button" tabindex="0"
     onclick="pagar()"
     onkeydown="teclas(event)">
  Pagar S/ 89.90
</div>
<script>
function teclas(e) {
  if (!['Enter', ' '].includes(e.key)) return;
  e.preventDefault(); pagar();
}
</script>
```

Imita a medias: sin `disabled` ni envío propio

El nativo: <button>

```
<form action="/pagar" method="post">
  <!— correo, tarjeta… —>
  <button type="submit">
    Pagar S/ 89.90
  </button>
</form>
```

Foco, Enter, Espacio, rol, `disabled` y envío: gratis.
Los estilos, con tu CSS

Matriz de decisión: cómo hacer el botón «Pagar»

	<div onclick>	div + role + JS	<button>
Funciona con teclado	No: Tab lo salta	Sí, si no olvidas Espacio	Sí, sin código extra
El lector dice «botón»	No	Sí	Sí
Envía el form y respeta `disabled`	Solo con tu JS	Solo con tu JS	Sí
Código que mantienes	Poco, pero roto	Mucho y frágil	Una etiqueta

Si se pulsa, es `<button>`. ARIA solo para lo que HTML no trae (pestañas, combobox), con el patrón del APG.

Pruébalo en 5 minutos: antes de abrir el PR



La hámster de QA ya prueba así

1. Suelta el mouse	Recorre el checkout con Tab y Shift+Tab. **¿Ves el foco? ¿Llegas a «Pagar» y paga con Enter?**
2. Zoom al 200 %	Ctrl y +. **¿Algo se corta o queda tapado?**
3. Enciende un lector	NVDA (gratis, Windows), VoiceOver (Cmd+F5 en Mac) o TalkBack. **¿Dice «botón»?**
4. Pasa una auditoría	Lighthouse en Chrome DevTools o axe DevTools: **encuentran contraste y etiquetas, no todo.**

Lo que te llevas

- Si se pulsa, es `<button>`; si te lleva, `<a href>`
- Cada campo con su `<label>`, y el foco visible y sin tapar
- Apunta a WCAG 2.2 AA y prueba con Tab antes de cada PR

```
> norma: w3.org/TR/WCAG22
> guía rápida: w3.org/WAI/WCAG22/quickref
> patrones: w3.org/WAI/ARIA/apg
> repo: github.com/Chamba-Lab
```